



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Creating Spark Dataframe

You have already seen the requirement for creating a Dataframe using the San Francisco fire call response data set. The data set is already available in the Databricks community cloud, and it is made available for you by the Databricks for learning purposes. You can access it from the following location in your Databricks community cloud. There is no need to download the data from the source and bring it to the cloud. That part is already done for you.

Problem Statement

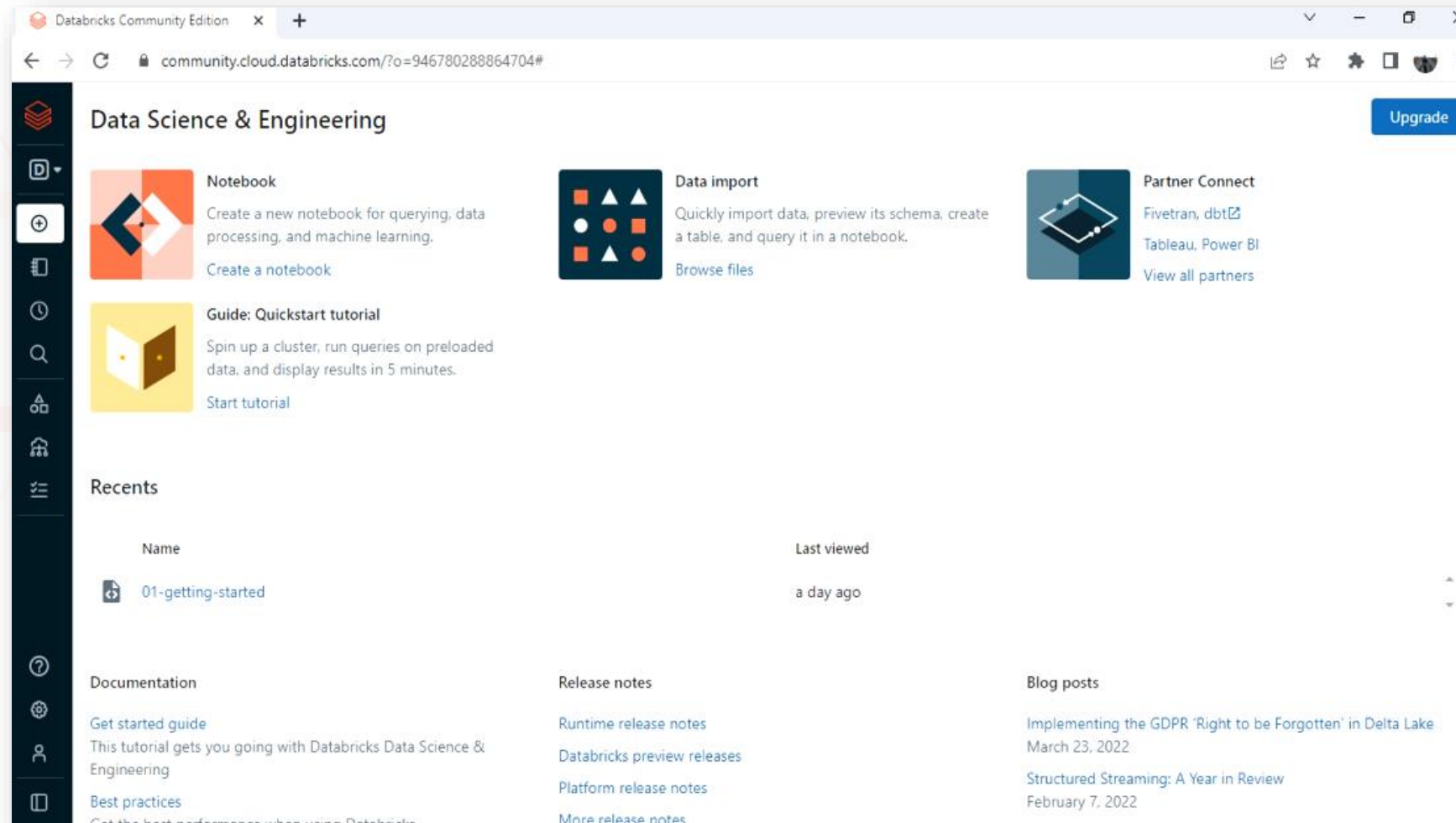
Data Set: Fire Department Calls for Service

Location: [/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv](#)

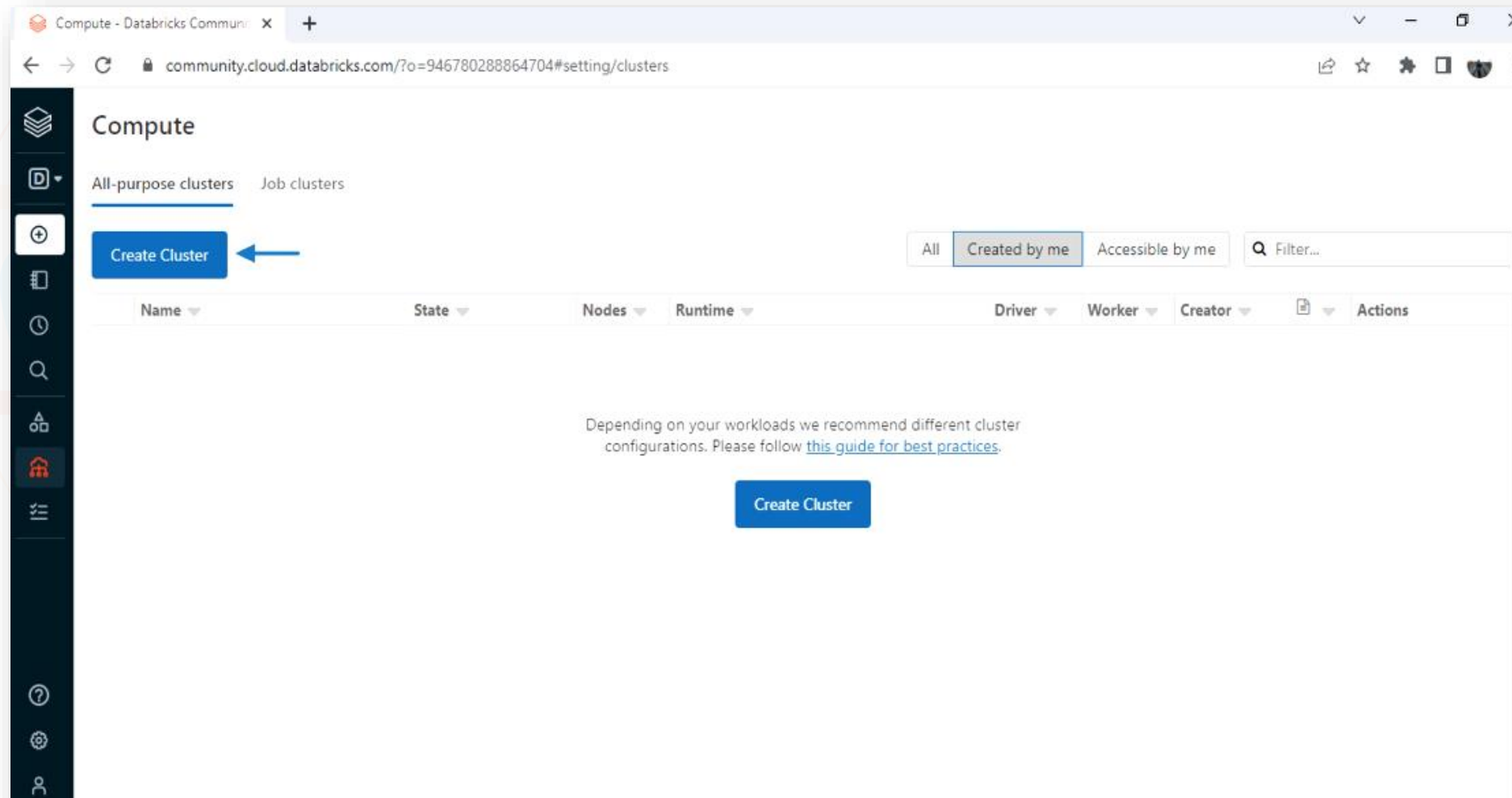
What are the requirements

1. Load the given data file and create a Spark data frame.
2. Use the Spark data frame to answer the following questions.
 1. How many distinct types of calls were made to the fire department?
 2. What are distinct types of calls made to the fire department?
 3. Find out all responses or delayed times greater than 5 mins?
 4. What were the most common call types?
 5. What zip codes accounted for the most common calls?
 6. What San Francisco neighborhoods are in the zip codes 94102 and 94103
 7. What was the sum of all calls, average, min, and max of the call response times?
 8. How many distinct years of data are in the CSV file?
 9. What week of the year in 2018 had the most fire calls?
 10. What neighborhoods in San Francisco had the worst response time in 2018?

Start your Databricks Workspace.



Go to the Compute menu and create a new cluster.



Give a name to your cluster and create it. Your cluster will start in a few minutes, and Databricks will terminate it if your cluster is idle for 2 hours. Once terminated, you cannot start it again. So if you want to work the next day, you should delete the older terminated cluster and create a new one. You can use your cluster for as long as you want and delete it when you are done.

Create Cluster - Databricks Com x +

community.cloud.databricks.com/?o=94678028864704#create/cluster

Create Cluster

New Cluster

Cancel Create Cluster

0 Workers: 0 GB Memory, 0 Cores, 0 DBU
1 Driver: 15.3 GB Memory, 2 Cores, 1 DBU

Cluster name

demo-cluster

Databricks runtime version

Runtime: 9.1 LTS (Scala 2.12, Spark 3.1.2)

Instance

Free 15 GB Memory: As a Community Edition user, your cluster will automatically terminate after an idle period of two hours. For more configuration options, please upgrade your Databricks subscription.

Instances Spark

Availability zone

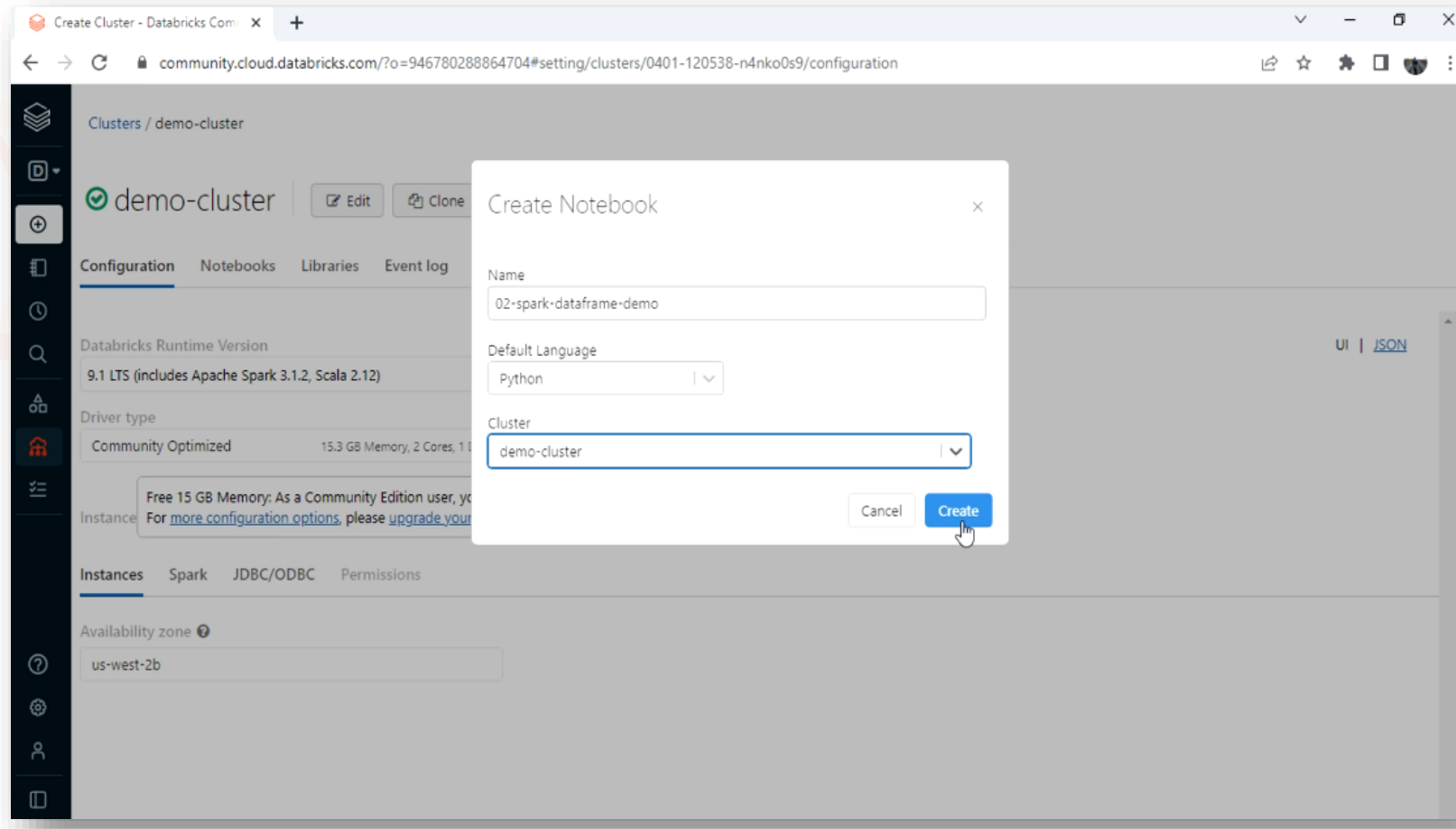
auto

Once your cluster is up and running, you can go ahead to the “*create*” menu options and create a new notebook.

Give a name to the notebook. Every notebook should have a default language.

You have four language options: Python, Scala, SQL and R.

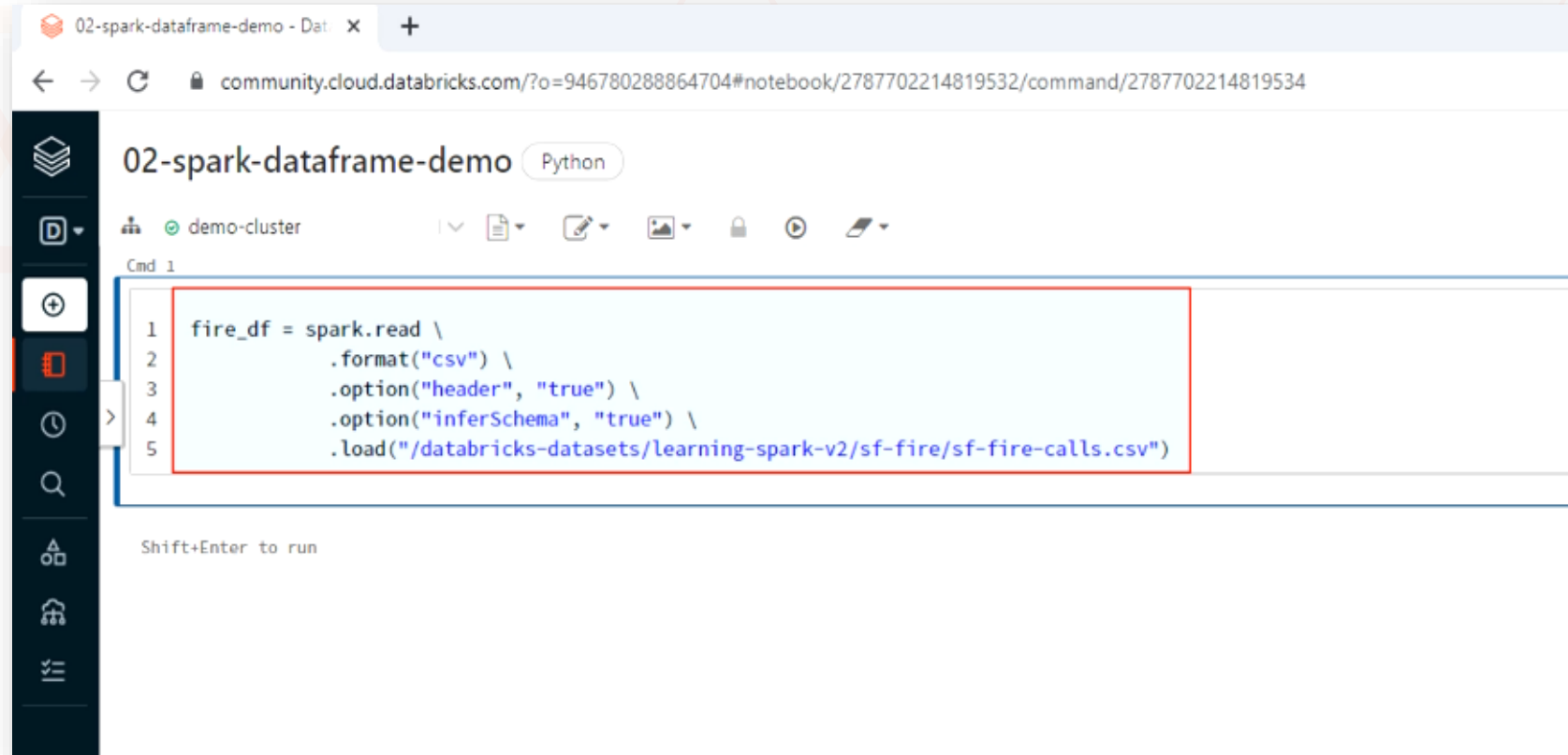
We want Python for Spark programming, so keep the default language Python. Your notebook will not run without a cluster. So you should also attach your cluster with the notebook. And finally, hit the create button.



Once the notebook is created, you can write the code to create a Spark Dataframe using the San Francisco fire call response data set.

And here we have the code for the same shown below. **Reference: (02-spark-dataframe-demo.ipynb)**

I am using spark.read which is a Spark Session object. It is the entry point for the Spark programming APIs. The rest of the code, such as format, option, and load, are the methods of the DataframeReader. I am telling the DataframeReader that the data file is a CSV file. Then I am telling that the CSV file comes with a header row. I am also saying that the DataframeReader should infer schema from the file itself. Finally, I call the load method so the DataframeReader can load the file from the given location. The code will return a dataframe, and I am assigning the dataframe to the fire_df variable.



The screenshot shows a Databricks notebook interface. The title bar indicates the notebook is named '02-spark-dataframe-demo' and is using Python. The URL in the browser is 'community.cloud.databricks.com/?o=946780288864704#notebook/2787702214819532/command/2787702214819534'. The notebook is running on a cluster named 'demo-cluster'. The code cell, labeled 'Cmd 1', contains the following Python code:

```
1 fire_df = spark.read \  
2     .format("csv") \  
3     .option("header", "true") \  
4     .option("inferSchema", "true") \  
5     .load("/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv")
```

Below the code cell, there is a prompt 'Shift+Enter to run'.

Visit <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql.html> to go to the Spark SQL documentation. You will see that all the things listed in this page belongs to the 14 objects in the pyspark.sql package which are listed in the image below. Each object offers some attributes and methods.

The screenshot shows the Apache Spark documentation page for Spark SQL. The page title is "Spark SQL" and the subtitle is "Core Classes". A list of 14 core classes is displayed, each with a brief description. The classes are:

1. `pyspark.sql.Session`
2. `pyspark.sql.DataFrame`
3. `pyspark.sql.DataFrameReader`
4. `pyspark.sql.DataFrameWriter`
5. `pyspark.sql.Row`
6. `pyspark.sql.Column`
7. `pyspark.sql.types`
8. `pyspark.sql.GroupedData`
9. `pyspark.sql.Window`
10. `pyspark.sql.Catalog`
11. `pyspark.sql.conf`
12. `pyspark.sql.functions`
13. `pyspark.sql.DataFrameNaFunctions`
14. `pyspark.sql.DataFrameStatFunctions`

The page also includes a sidebar with a search bar and a list of links to other documentation pages. The main content area is titled "Spark SQL" and "Core Classes".

Visit <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.DataFrame.html> to go to the DataFrameReader object under the Spark SQL documentation. The DataFrameReader offers eleven methods as shown in the screenshot below.

The screenshot shows the Apache Spark API documentation for the `DataFrameReader` class. The page title is "pyspark.sql.DataFrameReader" and the URL is "https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.sql.DataFrameReader.html". The page is divided into two main sections: "Attributes" and "Methods". The "Attributes" section lists several attributes with their descriptions. The "Methods" section lists 11 methods, which are highlighted in a red box. The methods are:

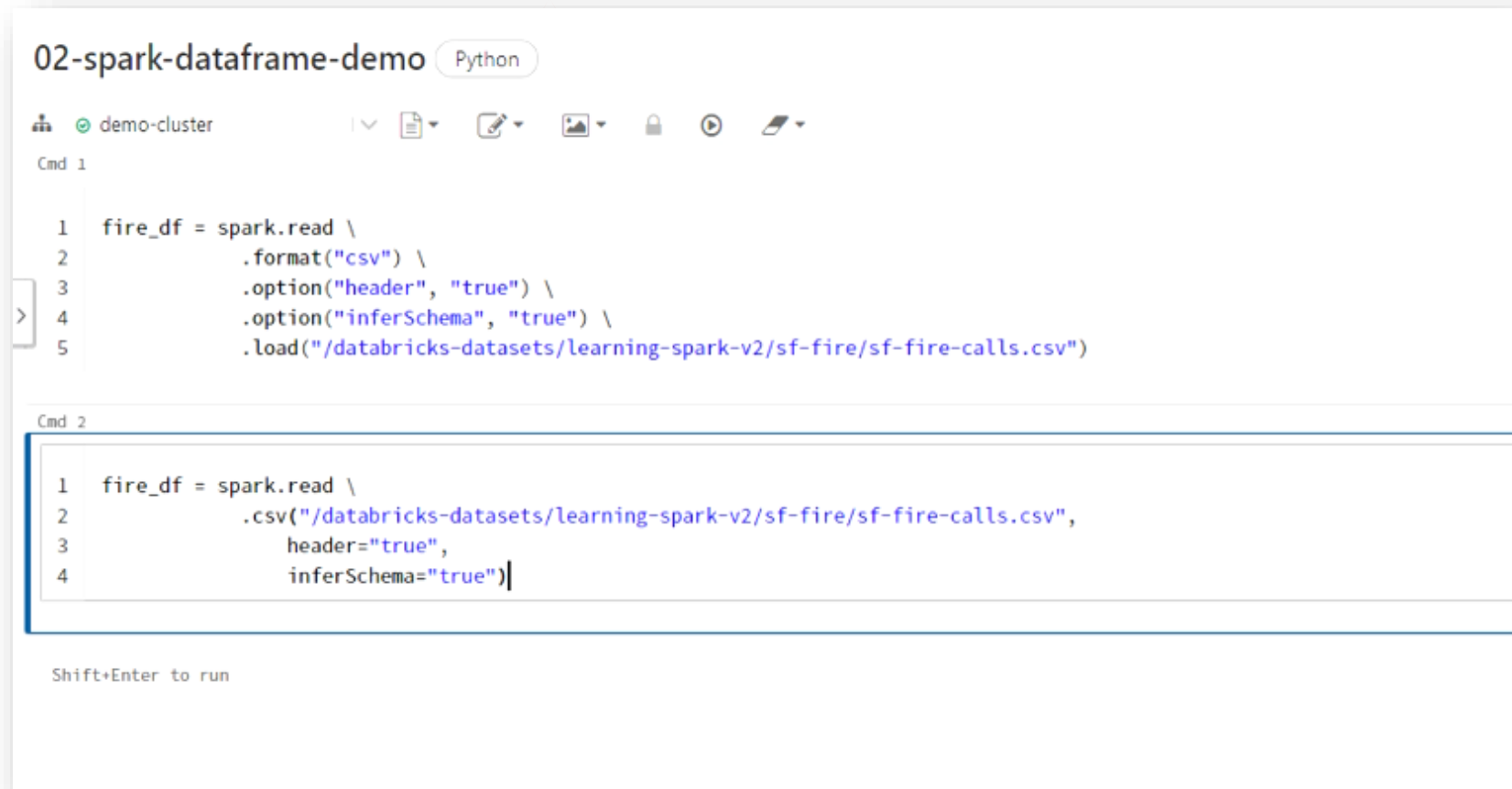
- 1. `DataFrameReader.csv`
- 2. `DataFrameReader.json`
- 3. `DataFrameReader.parquet`
- 4. `DataFrameReader.orc`
- 5. `DataFrameReader.jdbc`
- 6. `DataFrameReader.table`
- 7. `DataFrameReader.format`
- 8. `DataFrameReader.option`
- 9. `DataFrameReader.options`
- 10. `DataFrameReader.schema`
- 11. `DataFrameReader.load`

We can also create the same Dataframe using CSV method as shown in the second cell of the screenshot. In the first cell, I am using the format method to tell that I want to load a CSV file and the option method to set other configurations.

In the second cell, we are using the shortcut CSV method. I am not setting the format and options.

The CSV method implicitly tells that the format is CSV, and all other details will go as function arguments.

So, both the code are doing the same thing, but I prefer the first one as this code is standardized for loading data from any file format.



The screenshot shows a Databricks notebook titled "02-spark-dataframe-demo" in Python. It contains two code cells, "Cmd 1" and "Cmd 2", both of which load a CSV file from the Databricks dataset repository into a Spark DataFrame named "fire_df".

Cmd 1 (Standardized method):

```
1 fire_df = spark.read \  
2     .format("csv") \  
3     .option("header", "true") \  
4     .option("inferSchema", "true") \  
5     .load("/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv")
```

Cmd 2 (Shortcut CSV method):

```
1 fire_df = spark.read \  
2     .csv("/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv", \  
3         header="true", \  
4         inferSchema="true")
```

Below the code cells, it says "Shift+Enter to run".

I created a DataFrame, and now I want to see some records from this dataframe. You can use the show method. I take this data frame variable and call the Dataframe show method. I want to see ten records, so I use show(10). And you can see the output below the code cell. But it doesn't look nice and easy to read.

02-spark-dataframe-demo Python

demo-cluster

1 fire_df.show(10)

(1) Spark Jobs

[Call Number Unit ID Incident Number	CallType	Call Date	Watch Date	Call Final Disposition	Available DtTm	Address City Zipcode of					
Incident Battalion Station Area	Box OrigPriority Priority Final Priority ALS Unit Call Type Group NumAlarms	UnitType Unit sequence in call dispatch Fire	Prevention District Supervisor District	Neighborhood	Location	RowID	Delay				
20110014	M29	2003234	Medical Incident	01/11/2002 01/10/2002	Other 01/11/2002 01:58:...	10TH ST/MARKET ST SF					
94103	B02	36 2338	1	1	2	true	null	1	MEDIC		1
2		6	Tenderloin	(37.7765408927183...	020110014-M29	5.233333333333333					
20110015	M08	2003233	Medical Incident	01/11/2002 01/10/2002	Other 01/11/2002 02:10:...	300 Block of 5TH ST SF					
94107	B03	08 2243	1	1	2	true	null	1	MEDIC		1
3		6	South of Market	(37.7792841462441...	020110015-M08	3.083333333333333					
20110016	B02	2003235	Structure Fire	01/11/2002 01/10/2002	Other 01/11/2002 01:47:...	2000 Block of CAL... SF					
94109	B04	38 3362	3	3	3	false	null	1	CHIEF		6
4		5	Pacific Heights	(37.7895840679362...	020110016-B02	3.05					
20110016	B04	2003235	Structure Fire	01/11/2002 01/10/2002	Other 01/11/2002 01:51:...	2000 Block of CAL... SF					
94109	B04	38 3362	3	3	3	false	null	1	CHIEF		3
4		5	Pacific Heights	(37.7895840679362...	020110016-B04	2.316666666666667					

Command took 1.11 seconds -- by prashant@scholarnest.com at 4/1/2022, 8:18:39 PM on demo-cluster

So, you can use `display()` function. And now the output looks much better and easy to read. The `display` function is not part of Apache Spark. It is an additional tool by Databricks to format the output of a DataFrame. So it takes a DataFrame and shows the content of the data frame.

02-spark-dataframe-demo Python

demo-cluster 4 | 51Pacific Heights| (37.7895840679362...|020110016-B04| 2.316666666666667|

Command took 1.11 seconds -- by prashant@scholarnest.com at 4/1/2022, 8:18:39 PM on demo-cluster

Cmd 3

```
1 display(fire_df)
```

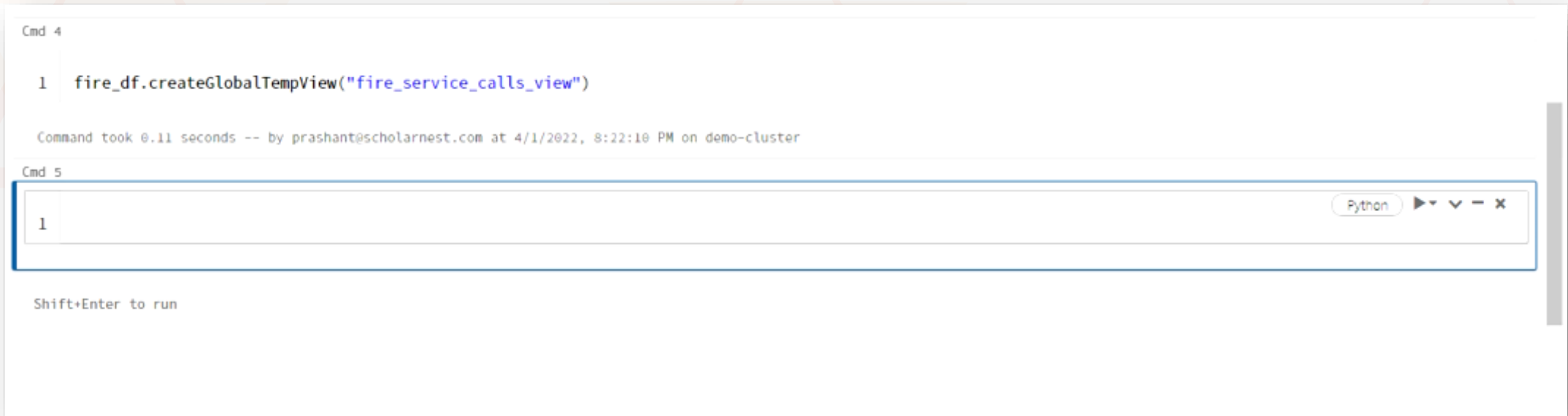
▶ (1) Spark Jobs

	Call Number	Unit ID	Incident Number	CallType	Call Date	Watch Date	Call Final Disposition	Available DtTm	Add
1	20110014	M29	2003234	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 01:58:43 AM	10TH
2	20110015	M08	2003233	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 02:10:17 AM	300
3	20110016	B02	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
4	20110016	B04	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:54 AM	200C
5	20110016	D2	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
6	20110016	E03	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
7	20110016	E38	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:17 AM	200C

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Command took 0.72 seconds -- by prashant@scholarnest.com at 4/1/2022, 8:20:12 PM on demo-cluster

In the code shown below, I am taking my dataframe and creating a global-temporary view for the dataframe. The view name is *fire_service_calls_view*. I am not converting my Dataframe to a table. Instead, I am creating a global-temporary view that is similar to the table.



```
Cmd 4

1 fire_df.createGlobalTempView("fire_service_calls_view")

Command took 0.11 seconds -- by prashant@scholarnest.com at 4/1/2022, 8:22:10 PM on demo-cluster

Cmd 5

1

Python ▶ ▾ - ✕

Shift+Enter to run
```

You can run SQL expression on the global-temporary view. But you need to change the cell type from Python to SQL. The global_temp is a hidden database, and all global temporary tables are created in this global_temp database. So you must use global_temp for accessing a global-temporary view.

02-spark-dataframe-demo Python

demo-cluster

Command took 0.11 seconds -- by prashant@scholarnest.com at 4/1/2022, 8:22:10 PM on demo-cluster

Cmd 5

```
1 %sql
2 select * from global_temp.fire_service_calls_view
```

▶ (1) Spark Jobs

	Call Number	Unit ID	Incident Number	CallType	Call Date	Watch Date	Call Final Disposition	Available DtTm	Add
1	20110014	M29	2003234	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 01:58:43 AM	10TH
2	20110015	M08	2003233	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 02:10:17 AM	300
3	20110016	B02	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
4	20110016	B04	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:54 AM	200C
5	20110016	D2	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
6	20110016	E03	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
7	20110016	E38	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:17 AM	200C

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)

Command took 0.96 seconds -- by prashant@scholarnest.com at 4/1/2022, 8:23:19 PM on demo-cluster

Cmd 6

```
1
```

Python



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com